Project Brief: Dodgers Rotation Pitch Arsenal Classifier

A summer data science portfolio project

The Question

Can a machine learning model identify what pitch was thrown using only the physical characteristics of the ball in flight — the same information a hitter has to read in about 400 milliseconds?

That's the whole project in one sentence. Everything below serves answering it well and telling the story clearly.

Why This Project Works

- **Real data, free.** MLB's Statcast system tracks every pitch thrown. The `pybaseball` Python library pulls it cleanly. No scraping, no paywalls, no permission needed.
- **Intuitive domain.** Anyone who's watched baseball understands the goal. That makes the writeup easy to follow, which is what a hiring manager or admissions reviewer cares about.
- **Right-sized.** It exercises the full data science workflow — acquisition, cleaning, exploration, modeling, evaluation, visualization — without requiring anything beyond Python, pandas, and scikit-learn.
- **A built-in "aha" moment.** When the model correctly separates a slider from a curveball using nothing but spin and movement, that's a satisfying result worth writing about.

Scope

In scope: The 2024 Los Angeles Dodgers starting rotation. Pick 3–5 starters with distinct arsenals so the data has variety to learn from. (Hint: a pitcher who throws four different pitches teaches the model more than one who throws two.)

Out of scope (for now): Every pitcher in baseball, real-time prediction, anything involving deep learning. Keep it tight. Scope creep is the most common reason summer projects never get finished.

Milestones

Milestone 1 — Data & Exploration (target: end of week 1)

Goal: Get the data, understand it, and prove you can describe what's in it.

Tasks:

- Figure out how to pull pitch-by-pitch Statcast data for your chosen pitchers.
- Identify which columns describe the *physical* characteristics of a pitch versus the *outcome* of a pitch. This distinction matters more than it first appears — think about why you'd want to keep them separate.
- Handle the messy parts: missing values, pitches with too few examples to be useful, and any columns that are essentially duplicates of each other.
- Produce distribution plots for the key physical features, broken out by pitch type.

Deliverable: A clean exploratory notebook. By the end you should be able to answer: which two features, on their own, do the best job of separating pitch types? Look at the plots before you build any model — you'll often see the answer with your eyes first.

Checkpoint question to answer in writing: What does the data actually contain, and what did you have to clean before it was usable?

Milestone 2 — The Classifier (target: end of week 2)

Goal: Train a model that predicts pitch type from physical features, and evaluate it honestly.

Tasks:

- Decide which features go into the model and which don't. (Reconsider that physical-vs-outcome distinction from Milestone 1. If you let the model "see" the outcome, are you really answering the question you set out to answer?)
- Split your data properly into training and testing sets. Understand *why* you can't evaluate a model on the same data you trained it on.

- Train a classification model. A Random Forest is a strong starting point — research why it suits this kind of data and what its tradeoffs are versus simpler alternatives.
- Evaluate with a confusion matrix, not just an accuracy number. The confusion matrix is where the interesting findings live.

Deliverable: A modeling notebook with a confusion matrix and a short interpretation.

Checkpoint question to answer in writing: Which pitches does the model confuse with each other? Form a hypothesis about *why* before you look it up. (When two pitches have similar spin and movement, even big-league hitters get fooled — does your model fail in the same places humans do?)

Milestone 3 — Visualization & Story (target: end of week 3)

Goal: Turn the work into something someone would want to read.

Tasks:

- Build the signature plot of pitch analytics: horizontal movement versus vertical movement, with each point colored by pitch type. Make one per pitcher. These are the visuals that make the project look professional — invest time in making them clean.
- Write the narrative. Pick one pitcher with an unusual or elite pitch and explain what makes it different, using your own plots as evidence.

Deliverable: A visualization notebook and a written summary a non-technical reader could enjoy.

Stretch Goal (optional — adds "Tier 2" depth)

Bring the hitter back into the picture. Do the pitches the *model* finds hard to classify also tend to generate more swings-and-misses or weak contact? If a pitch is physically ambiguous to a machine, is it also deceptive to a human?

This turns a classification exercise into a genuine research question with a real answer sitting in the data. It's the kind of thing that makes a portfolio project memorable — but only attempt it after the core three milestones are solid. A finished simple project beats an unfinished ambitious one every time.

Tools

- **Python** with **pandas** for data work
- **scikit-learn** for the model
- **matplotlib** / **seaborn** for visuals
- **pybaseball** for the data
- **Jupyter notebooks** for the deliverables
- **Git / GitHub** for version control — start the repo on day one, commit often

---

Repository Structure (what a reviewer opens first)

A clean repo signals a clean thinker. Aim for something like:

- A **README** that tells the story in one page: the question, the approach, the headline finding, and one great visual. This is the single most important file. Most reviewers decide whether to keep reading based on the README alone.
- A **data** folder (and learn why raw data often shouldn't be committed to Git — research `.gitignore`).
- Separate, clearly named **notebooks** for exploration, modeling, and visualization.
- A **requirements** file so someone else can reproduce your environment.

How to Know You're Done

You're finished when you can hand the repo to someone who's never seen it and they can, in five minutes: understand the question, see the answer, and trust that you did the work carefully. That standard — reproducible, legible, honestly evaluated — is what separates a portfolio project from a homework assignment.

A Note on Doing It Yourself

The temptation will be to ask an AI to write the code and move on. Resist it. The entire value of this project — for both your learning and your portfolio — is in the decisions you make and the problems you work through yourself. Use documentation, use Stack Overflow, use the library's own examples. When you get stuck, sit with it for a while before reaching for help. The stuck part is where the learning happens, and it's also the part you'll be proud to talk about in an interview.